



نظري

4

13

2340 sp

كلية الهندسة المعلوماتية

السنة الثانية

Packages and Interfaces

د.غاندي هسام



برمجة ٣

RB Informatics; 30/10/2025

المهارات والمفاهيم الأساسية في java

- استخدام الحزم (Packages)
- فهم كيفية تأثير الحزم على الوصول (Access)
- تطبيق معدل الوصول المحمي (Protected Access Modifier)
- استيراد الحزم
- معرفة الحزم القياسية في Java
- فهم أساسيات الواجهات (Interfaces)
- تنفيذ واجهة (Implement an Interface)
- استخدام مراجع الواجهة (Interface References)
- فهم متغيرات الواجهة (Interface Variables)
- توسيع الواجهات (Extend Interfaces)
- إنشاء طرق افتراضية ثابتة و خاصة

Packages

تعريف ال package:

- ال package هي آلية تُستخدم لتجميع مجموعة من الأصناف (classes) والحزم الفرعية (sub-packages) والواجهات (interfaces) ذات العلاقة ضمن إطار تنظيمي واحد.

الوظيفتين الأساسيتين للحزم :

1. التنظيم : الحزم تمنح المشروع بنية واضحة ومرئية فبدلاً من وضع مئات الأصناف في مكان واحد نضع الأصناف التي تعرض المعلومات وتتفاعل مع المستخدم في حزمة ونضع الأصناف التي تقوم بالعمليات الحسابية او معالجة البيانات في حزمة أخرى كمثال مما يمنع الفوضى في الكود وتمنحك سهولة فائقة في تطوير المشروع وتحديد الأخطاء و إصلاحها
2. التحكم بالوصول: الحزم تشكل حدود او نطاق لتحديد مدى رؤية الأصناف ومكوناتها لبعضها البعض وهي أساسية لتطبيق مبدأ التغليف في ال oop

مفهوم الحزمة الافتراضي The Default package

- في حال لم تكتب كلمة package في بنية الملف فان الجافا تضع الكلاس تلقائيا ضمن الحزمة الافتراضية التي ليس لها اسم فتبدو وكأنها شفافة او غير موجودة

- ملاحظة 😊:** تجنب استخدام Default package في المشاريع الكبيرة لان العناصر الموجودة في الحزمة الافتراضية لا يمكن الوصول اليها من أي حزمة مسماة أخرى باستخدام امر import لذلك من المفضل استخدامها فقط للبرامج البسيطة

كيفية تعريف الحزمة :

- حتى تنشئ حزمة نكتب الكلمة المفتاحية package كاول سطر في ملف الجافا وتكون كل الكلاسات المعرفة بداخله تابعة له
- The .class files :**
- ملفات الجافا المجمعة التي تخص حزمة معينة مثال myschool يجب ان تكون مخزنة في مجلد يحمل نفس الاسم بالضبط لان جافا عندما بتشغيل او استيراد الكلاس فانها تتوقع العثور عليه في مجلد يطابق اسمه في حال لم تجده هناك يفشل البرنامج
- الحساسية لحالة الاحرف : أسماء الحزم في جافا حساسة لحالة الاحرف اذا كان الكود موجود في حزمة MyPack; package فان اسم المجلد يجب ان يكون MyPack وليس mypack مثلا (حرف صغير).

هيكلية الحزم الهرمية:

- الحزم ليست مجلدات فردية فقط بل يمكن ان تكون هرمية (متداخلة) يتم استخدام النقطة(.) كفاصل لتمثيل التداخل أي كل جزء مفصول بنقطة يمثل مجلد فرعي
- الصيغة العامة لهرمية الحزم :
- Package pack1.pack2;
- هذا يعني انك تستطيع تمديد اسم الحزمة الى أي عدد من المستويات التي يحتاجها لتنظيم مشروعك



"Your light is meant to be seen"

كيفية بحث نظام التشغيل عن مكان الحزم :

1. مجلد العمل الحالي : هذه هي الطريقة الأكثر بساطة عندما تشغل برنامج جافا فان النظام يبدأ عملية البحث عن ملفات الكلاس من هذا المجلد تحديدا
2. متغير البيئة: يتم اعداده في نظام التشغيل وظيفته اخبار نظام تشغيل جافا بقائمة ثابتة من المسارات التي يجب ان يبحث فيها عن ملفات الكلاس
3. خيار سطر الأوامر : هو خيار يتم اضافته مباشرة الى امر التشغيل او التجميع يسمح لك بتحديد المسارات المطلوبة للتشغيل او التجميع الحالي
4. مسار الوحدة : هو قائمة بالمواقع التي تبحث فيها آلة جافا الافتراضية عن الوحدات المطلوبة لتشغيل او ترجمة مشروعك وقد ظهر مع الإصدار التاسع لجافا

مثال :

`package backpack;` ← This file is part of the backpack package.

`class Book {` ← thus, Book is part of backpack.

```
private String title;
private String author;
private int pubDate;
```

```
Book(String t, String a, int d) {
    title = t;
    author = a;
    pubDate = d;
}
```

```
void show() {
    System.out.println(title);
    System.out.println(author);
    System.out.println(pubDate);
    System.out.println();
}
```



```

class BookDemo { ← BookDemo is also part of backpack
    public static void main(String[] args) {
        Book[] books = new Book[5];

        books[0] = new Book("Java: A Beginner's Guide", "Schildt", 2022);
        books[1] = new Book("Java: The Complete Reference", "Schildt", 2022);
        books[2] = new Book("1984", "Orwell", 1994);
        books[3] = new Book("Red Storm Rising", "Clancy", 1986);
        books[4] = new Book("On the Road", "Kerouac", 1955);

        for (int i = 0; i < books.length; i++) {
            books[i].show();
        }
    }
}

```

Access Modifier:

1. Default Access, package private :

الاصناف التي تعرف ضمن حزمة يمكن جعلها خاصة بتلك الحزمة فقط بحيث لا يمكن للتعليمات البرمجية خارج الحزمة المحددة الوصول اليها او استخدامها هذا ما يسمى ب Default Access او package private يحدث عندما لا تكتب أي محدد وصول Access Modifier صريح مثل public او private

2. الوصول العام Public Access :

هو أوسع نطاق للوصول أي عضو (متغير، دالة) معلن ك public هو مرئي ومتاح للاستخدام في أي مكان داخل المشروع لا يهم ما اذا كان الكلاس المستخدم يقع في نفس الحزمة او في حزمة مختلفة

3. الوصول الخاص private Access :

هو أضيق نطاق للوصول وأقوى شكل من اشكال التغليف العضو المعلن ك private لا يمكن الوصول اليه واستخدامه الا من داخل الكلاس نفسه الذي تم تعريفه منه

■ ملاحظة هامة جدا :

■ مستوى الوصول الخاص هو الوحيد الذي لا يكثرث للحزم سواء كان الكلاس المستخدم من نفس الحزمة او حزمة مختلفة طالما انه ليس هو الكلاس الأصلي فان الوصول اليه سيكون ممنوع وهذا يضمن اعلى مستوى من الأمان والتحكم في الكلاس

4. الوصول المحمي protected:

هذا المستوى يوفر توازن بين التغليف و مرونة الوراثة يمكن لأي عضو معلن ك protected ان يستخدم في نطاقين رئيسيين

أولا داخل الحزمة : أي كلاس آخر يقع ضمن نفس الحزمة يمكنه الوصول الى هذا العضو .

ثانيا : كلاسات الأبناء في أي مكان اذا كان هناك كلاس وارث يقع في حزمة مختلفة فان كلاس الابن هذا يمكنه الوصول الى العضو المحمي المورث من كلاس الاب

	Private Member	Default Member	Protected Member	Public Member
Visible within same class	Yes	Yes	Yes	Yes
Visible within same package by subclass	No	Yes	Yes	Yes
Visible within same package by non_subclass	No	Yes	Yes	Yes
Visible within different package By subclass	No	No	Yes	Yes
Visible within different package by non_subclass	No	No	No	Yes

مثال

```
package backpack;
```

```
public class Book {
```

Book and its members must be public in order to be used by other packages

```
    private String title;
    private String author;
    private int pubDate;
```

```
    public Book(String t, String a, int d) {
        title = t;
        author = a;
        pubDate = d;
    }
```

```
    public void show() {
        System.out.println(title);
        System.out.println(author);
        System.out.println(pubDate);
        System.out.println();
    }
```

// Use the Book class from backpack.

```
class UseBook {
    public static void main(String[] args) {
        backpack.Book[] books = new backpack.Book[5];

        books[0] = new backpack.Book("Java: A Beginner's Guide", "Schildt", 2022);
        books[1] = new backpack.Book("Java: The Complete Reference", "Schildt", 2022);
        books[2] = new backpack.Book("1984", "Orwell", 1949);
        books[3] = new backpack.Book("Red Storm Rising", "Clancy", 1986);
        books[4] = new backpack.Book("On the Road", "Kerouac", 1955);

        for(int i = 0; i < books.length; i++) books[i].show();
    }
}
```

quality Book with its package name :backpack.

▪ الكلاس عالي المستوى Top-level class :

هو الكلاس الذي يتم تعريفه داخل ملف الجافا وليس داخل كلاس اخر أي ليس كلاس داخلي لا يمكن استخدام private او protected لكلاس عالي المستوى لان الكلاس private يجب ان يكون جزء من كلاس اخر والكلاس protected يستخدم عادة للوراثة بين الحزم

▪ استيراد الحزم :

الطريقة الأساسية لاستخدام أي كلاس لا ينتمي الى حزمته الحالية هي باستخدام الاسم المؤهل بالكامل هذا يعني انك تكتب اسم الحزمة متبوعا بنقطة ثم اسم الكلاس هذه الطريقة تضمن لجافا معرفة مكان الكلاس بالتحديد لتجنب أي التباس اذا كان هناك كلاس اخر يحمل نفس الاسم في حزمة أخرى فيما بعد تم استخدام الكلمة المفتاحية import لتجنب كتابة الاسم المؤهل بالكامل في كل مرة توضع في بداية ملف الجافا الخاص بك

▪ الشكل العام لجمله import :

نضع الكلمة المفتاحية import متبوعا بالاسم الكامل للحزمة والكلاس الذي تريد استخدامه

```
Import package1.classname;
```

اذا كنت تخطط لاستخدام العديد من الكلاسات داخل حزمة معينة وليس كلاس واحد فقط يمكنك استخدام علامة النجمة (*) بدلا من اسم الكلاس

```
import package.*;
```

```

package bookpackext;
import bookpack.*; ← import bookpack.

// Use the Book class from bookpack.
class UseBook {
    public static void main(String[] args) {
        Book[] books = new Book[5];

        books[0] = new Book("Java: A Beginner's Guide", "Schildt", 2022);
        books[1] = new Book("Java: The Complete Reference", "Schildt", 2022);
        books[2] = new Book("1984", "Orwell", 1949);
        books[3] = new Book("Red Storm Rising", "Clancy", 1986);
        books[4] = new Book("On the Road", "Kerouac", 1955);

        for(int i=0; i < books.length; i++) books[i].show();
    }
}

```

■ مكتبة كلاسات جافا :

■ **الفئات القياسية :** لغة جافا أتت مع مجموعة ضخمة ومصممة مسبقا من الكلاسات القياسية هذه الكلاسات توفر وظائف أساسية مثل التعامل مع الادخال والإخراج التاريخ الوقت وغيرها هذه الكلاسات متاحة لاي برنامج جافا ترغب في كتابته

■ يشار الى مكتبة الكلاسات هذه بشكل شائع باسم واجهة برمجة تطبيقات جافا API

■ مفهوم API :

■ مجموعة من القواعد والادوات التي تسمح للمبرمجين بإنشاء تطبيقات برمجية وهو في سياق جافا يمثل كل الكلاسات والدوال التي توفرها اللغة لتمكينك من بناء برامجك

■ java.lang package :

■ كل برامج جافا التي تم كتابتها والعمل بها تعتمد بشكل ضمني على الحزمة المسماة java.lang تحتوي هذه الحزمة على الكلاسات الأساسية والضرورية التي لا يمكن لاي برنامج ان يعمل بدونها

■ من بين الفئات العديدة و الأساسية الموجودة داخل java.lang هناك كلاس مهم جدا يسمى system حيث يحتوي على كائنات جاهزة للاستخدام و ابرزها هو الكائن الزني يستخدم في عمليات الإخراج out

■ حزمة java.lang هي الحزمة الوحيدة التي يقوم مترجم جافا باستيرادها تلقائيا في كل ملف نقوم بإنشائه

interface

- هي مفهوم أساسي في ال oop وهي اشبه بالعقد contract او المخطط بهدف تحديد ماذا يجب على الكلاس فعله دون تحديد كيفية القيام بذلك لقد راينا مثال لهذا المفهوم في التابع المجرد الذي يعرف كتوقيع (اسم, معاملات , نوع الارجاع) بدون أي تنفيذ او جسم له
- الكلاس الفرعي الذي يرث من كلاس اعلى يحتوي على توابع مجردة يجب عليه ان ينفذ كل التوابع المجردة الموجودة فيه
- الواجهة هي امتداد لمفهوم التابع المجرد فعند تعريف واجهة ما يمكن لأي عدد من الكاسات ان ينفذها والاهم هو ان الكلاس الواحد يمكنه تنفيذ أي عدد من الواجهات على عكس الوراثة حيث يتم الوراثة من كلاس واحد فقط

الوصول الى الواجهات :

- بالنسبة للواجهات ذات المستوى الأعلى مستويان الوصول المتاحان هما public او بدون محدد أي default
- اذا لم يتم تضمين محدد وصول يتم تطبيق الوصول الافتراضي وهذا ما يجعل الواجهة متاحة فقط للاعضاء حزمته
- اذا تم الإعلان عن واجهة ك public يجب ان تكون موجودة في ملف جافا يحمل نفس الاسم
- التوابع المعلنة في الواجهة هي ضمناً عامة حتى لو لم تكتب كلمة public قبل التابع فان جافا تعتبره public والسبب هو ان الغرض من الواجهة هو تحديد عقد يجب ان ينفذ بشكل عام
- المتغيرات في الواجهة:
- أي متغير يتم الإعلان عنه داخل الواجهة ليس متغير كائن عادي ولكن ثابتة ذو خصائص معينة ولا يمكنك تغييرها

1. ليست متغيرات كائن لا تنتمي الى كائن محدد بل الى الواجهة نفسها

2. متاحة للجميع

3. Final ثابتة لايمكن تغيير قيمتها

4. يجب تهيئتها عند الإعلان عنها مباشرة

```
public interface Series {

    int getNext();//return next number in series

    void reset();//restart

    void setStart(int x);//set starting value
}
```



تنفيذ الواجهات :

لتجعل كلاس معين يلتزم ببروتوكول الواجهة يجب عليك إضافة الكلمة المفتاحية `implements` بعد اسم الكلاس مباشرة متبوع باسم الواجهة

يمكن لكلاس واحد ان ينفذ اكثر من واجهة في هذه الحالة يتم فصل أسماء الواجهات باستخدام فاصلة

`Class MyClass implements interface1,interface2;`

مثال : كلاس ينفذ واجهة series التي رايناها سابقا

```
class ByTwos implements Series {
    int start;
    int val;
    ByTwos() {
        start = 0;
        val = 0;
    }
    public int getNext() {
        val += 2;
        return val;
    }
    public void reset() {
        val = start;
    }
    public void setStart(int x) {
        start = x;
        val = x;
    }
}
```



Keep Going Muhandezzz

```
class ByThrees implements Series {
    int start;
    int val;
    ByThrees() {
        start = 0;
        val = 0;
    }
    public int getNext() {
        val += 3;
        return val;
    }
    public void reset() {
        val = start;
    }
    public void setStart(int x) {
        start = x;
        val = x;
    }
}
```

قواعد الوصول لتوابع التنفيذ :

التوابع التي تقوم بتنفيذ الواجهة المجردة يجب الإعلان عنها كـ `public` السبب هو ان جميع التوابع في الواجهة هي `public` ضمنا وعندما يقوم كلاس فرعي بتنفيذ تابع من واجهة لا يمكنه تقييد وصوله يجب ان يكون مستوى الوصول للتنفيذ مساويا او اوسع من مستوى وصول المحدد في الأصل

استخدام مراجع الواجهة :

يمكنك انشاء مرجع واجهة يشير الى أي كائن ينفذ تلك الواجهة مرجع الواجهة بدلا من تعريف متغير مرجعي باستخدام اسم فئة مثل

```
Class A obj = new classA;
```

يمكنك استخدام اسم الواجهة كنوع بيانات

مثال :

إذا كان لديك واجهة اسمها `Shape` ونفذتها كلاسات مثل `Circle, Square, Triangle`

فان مرجع الواجهة `Shape` يمكنه الإشارة الى كائن من أي من هذه الكلاسات

```
Shape s = new Circle();
```

هذا يسمح لك بكتابة كود عام لا يحتاج الى معرفة النوع الفعلي للكائن في وقت التصميم بل يتعامل معه بناء على العقد الذي تفرضه الواجهة

```

class SeriesDemo2 {
    public static void main(String[] args) {
        ByTwos twoOb = new ByTwos();
        ByThrees threeOb = new ByThrees();
        Series ob;
        for(int i = 0; i < 5; i++) {
            ob = twoOb;
            System.out.println("Next ByTwos value is " + ob.getNext());
            ob = threeOb;
            System.out.println("Next ByThrees value is " + ob.getNext());
        }
    }
}

```

- البرامج الكبيرة عادة ما تستخدم العديد من القيم الثابتة مثل حجم مصفوفة والحدود المختلفة والقيم الخاصة لذلك عندما يكون لديك ثوابت تحتاجها ككلاسات مختلفة قد يكون من غير العملي تعريف هذه الثوابت مرارا و تكرارا في كل فئة او تعريفها في فئة أساسية غير ذات صلة لحل هذه المشكلة نقوم بإنشاء واجهة تحوي كل هذه الثوابت فقط بدون توابع بمجرد التنفيذ يمكن للفئة الوصول الى الثوابت مباشرة باستخدام اسمها البسيط دون الحاجة لاستخدام اسم الواجهة

```

interface IConst {
    int MIN = 0;
    int MAX = 10;
    String ERRMSG = "Boundary Error";
}
class IConstD implements IConst {
    public static void main(String[] args) {
        int[] nums = new int[MAX];

        for (int i = MIN; i < 11; i++) {
            if (i >= MAX)
                System.out.println(ERRMSG);
            else {
                nums[i] = i;
                System.out.print(nums[i] + " ");
            }
        }
    }
}

```



التوابع الافتراضية للواجهة :

- قبل الإصدار الثامن لجافا لم يكن بإمكان الواجهة ان تعرف أي تنفيذ على الإطلاق كان كل تابع في الواجهة مجرد غير الإصدار الثامن هذا الامر عن طريق إضافة إمكانية جديدة للواجهة تسمى التابع الافتراضي أي اصبح تابع الواجهة يملك جسم وبذلك يوفر تنفيذ افتراضي
- كان الهدف الرئيسي لانشاء التابع الافتراضي هو توفير وسيلة يمكن من خلالها توسيع الواجهات (إضافة توابع جديدة لها) دون افساد الكود الموجود مسبقا

```
public interface MyIF {
    // This is a "normal" interface method declaration.
    // It does NOT define a default implementation.
    int getUserID();

    // This is a default method. Notice that it provides
    // a default implementation.
    default int getAdminID() {
        return 1;
    }
}
```

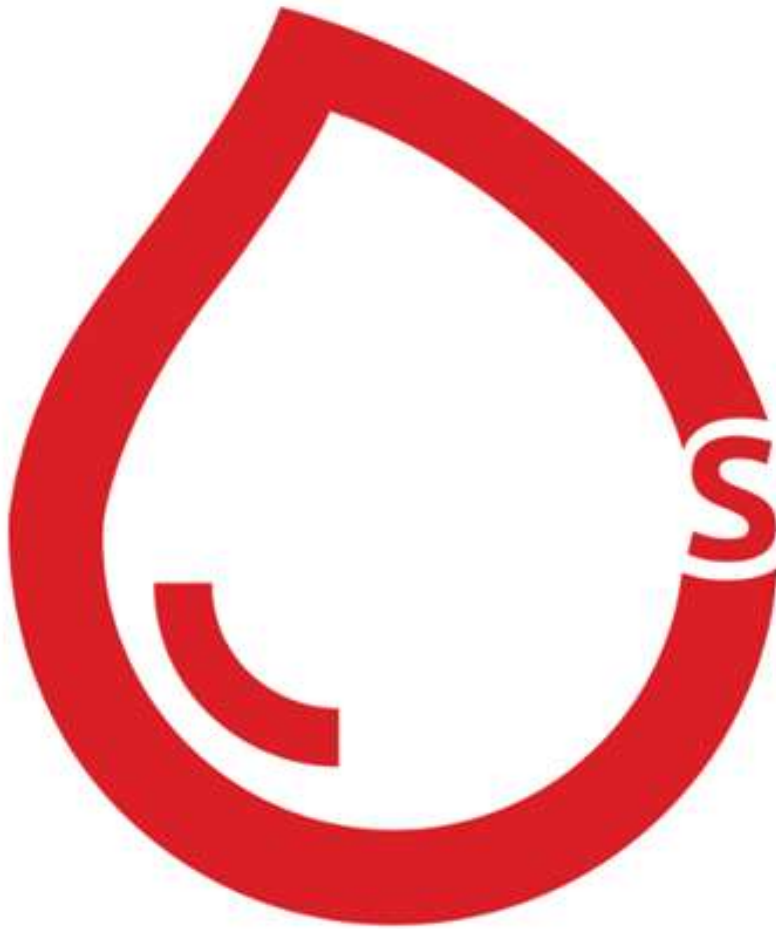
استخدام التوابع الساكنة داخل الواجهة :

- أضاف الإصدار الثامن إمكانية جديدة أخرى للواجهات وهي القدرة على تعريف واحد او اكثر من التوابع الساكنة static method حيث يتم استدعاؤه عن طريق تحديد اسم الواجهة وليس عن طريق كائن من كلاس ينفذ الواجهة السبب في ذلك ان التوابع الساكنة ترتبط بالكلاس او الواجهة نفسها وليست بالكائنات الفردية تستخدم عادة كادوات مساعدة في الواجهة

```
public interface MyIF {
    // This is a "normal" interface method declaration.
    // It does NOT define a default implementation.
    int getUserID();

    // This is a default method. Notice that it provides
    // a default implementation.
    default int getAdminID() {
        return 1;
    }

    // This is a static interface method.
    static int getUniversalID() {
        return 0;
    }
}
```



RBCs

